Module 4 -- Arduino Microcontroller

# Lesson 28: Reading Digital Inputs -- Buttons

★ Star-3 Intermediate ⏰ 40--45 minutes 👤 Age 6--12 years ⚡ 250 XP

## Your Mission Today

Circuit Explorer, so far your Arduino has only been SENDING signals out. Today it learns to LISTEN. You will connect a push button and teach your Arduino to detect when you press it. Press the button and the LED turns on. Release it and the LED turns off. Then you will level up with an **if statement** — the Arduino's way of making decisions. This is where your Arduino stops being a blinking toy and starts being a SMART gadget!

## Learning Objectives

By the end of this lesson, you will be able to:

- Wire a push button to an Arduino digital pin using INPUT_PULLUP
- Use `digitalRead()` to check if a button is pressed or not
- Write an `if/else` statement to make decisions in code
- Measure button pin voltage with the Wand (pressed vs not pressed)
- Print button state to the Serial Monitor

## What You Need

| Item | Qty |
|---|---|
| Arduino Uno + USB cable | 1 |
| Breadboard | 1 |
| Push button (tactile switch) | 1 |
| LED (any color) | 1 |
| 330-ohm resistor | 1 |
| Jumper wires | 5 |
| Multimeter (Magic Measurement Wand!) | 1 |

## How to Teach This Lesson

### Step 1: Hook — The Stubborn Light (3 min)

💬

"Right now, your Arduino blinks an LED on its own. But what if you wanted it to only turn on when you PRESS a button? Like a doorbell — press the button, the bell rings. Let go, it stops."

💬

"To do this, the Arduino needs to LISTEN to something in the real world. It needs an INPUT. A button is the simplest input there is."

**Award:** **+10 XP** **for understanding inputs vs outputs!**

## Step 2: How Buttons Work with Arduino (5 min)

**The Problem:**

> "A button either connects two wires together (pressed) or leaves them disconnected (released). But the Arduino needs a clear voltage — HIGH (5V) or LOW (0V). If the pin is just floating disconnected, it gets confused and reads random garbage."

**The Solution — Pull-Up Resistor:**

> "We use a special trick called INPUT_PULLUP. This tells the Arduino to use a built-in resistor that gently pulls the pin UP to 5V. When you press the button, it connects the pin to GND (0V), pulling it DOWN."

```
NOT PRESSED:                    PRESSED:
Pin 2 ---- [internal pullup]    Pin 2 ---- [internal pullup]
       \--- (button open)               \--- [button] --- GND
       reads HIGH (5V)                  reads LOW (0V)
```

> "It seems backwards! When the button is NOT pressed, the pin reads HIGH. When pressed, it reads LOW. Just remember: pressed = LOW with pullup."

**Award:** **+10 XP** **for understanding pull-up resistors!**

## Step 3: Build the Button Circuit (8 min)

**Wiring Diagram:**

```
Arduino Uno                    Breadboard
+---------+
| Pin 2   |----[wire]----+
|         |              |
|         |         [BUTTON] (one side)
|         |         [BUTTON] (other side)
|         |              |
| GND     |----[wire]----+
|         |
| Pin 7   |----[wire]---[330 ohm]---LED(+)---LED(-)---GND
+---------+

Button wiring detail:
- Button has 4 legs in a square
- Connect Pin 2 to one side
- Connect GND to the other side
- The button connects these when pressed
```

**Steps:**

1. Place the **push button** across the center gap of the breadboard
2. Connect one side of the button to **Arduino pin 2**
3. Connect the other side of the button to **Arduino GND**
4. Keep your LED from last lesson: pin 7 through a 330-ohm resistor to GND

**Award:** **+20 XP** **for wiring the button circuit!**

## Step 4: Write the Button Code (10 min)

**The If Statement:**

"An if statement is how the Arduino makes decisions. It is like a fork in the road: IF this is true, do THIS. ELSE (otherwise), do THAT."

```
// Lesson 28: Button Controls LED

int buttonPin = 2;
int ledPin = 7;

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);  // Button with pull-up resistor
  Serial.begin(9600);
  Serial.println("Button LED Ready!");
}

void loop() {
  int buttonState = digitalRead(buttonPin);  // Read the button

  if (buttonState == LOW) {    // LOW = pressed (with pullup)
    digitalWrite(ledPin, HIGH);  // Turn LED ON
    Serial.println("Button PRESSED -- LED ON");
  } else {
    digitalWrite(ledPin, LOW);   // Turn LED OFF
    Serial.println("Button RELEASED -- LED OFF");
  }
}
```

**Important Details:**

- `digitalRead(buttonPin)` reads the pin and returns either `HIGH` or `LOW`
- `==` is the "equals" comparison (two equal signs, not one!)
- The curly braces `{ }` contain the code that runs for each condition

**Upload and test!** Press the button — LED turns on. Release — LED turns off.

**Award:** **+30 XP** **for a working button-controlled LED!**

## Step 5: Wand Check — Measure Button Pin Voltage (8 min)

💬

"Let us see what the Magic Measurement Wand says about the button pin. This will prove that our pull-up resistor theory is correct!"

**Setup:**

1. Set the Wand to **DC Voltage**
2. Black probe on **GND**
3. Red probe on **pin 2** (the button pin)

**Measurements:**

```
| Button State    | Expected Voltage | Your Reading | Why? |
|----------------|-----------------|-------------|------|
| NOT pressed     | ~5.0V           |              | Pull-up holds pin HIGH |
| PRESSED         | ~0.0V           |              | Button connects to GND |
```

💬

> "See! When the button is not pressed, the internal pull-up resistor pulls pin 2 up to 5V. When you press the button, it connects pin 2 directly to GND, so it drops to 0V. The Arduino reads this change and responds!"

**Bonus — Measure while watching the Serial Monitor:**

💬

> "Press the button slowly and watch both the Wand AND the Serial Monitor at the same time. The moment the voltage drops, the Serial Monitor shows PRESSED. They match perfectly!"

**Award:** **+40 XP** **for measuring button pin voltage in both states!**

---

## Step 6: Challenge — Toggle Mode (8 min)

💬

> "Right now the LED is only on WHILE you hold the button. Can you make it TOGGLE — press once to turn on, press again to turn off?"

```
int buttonPin = 2;
int ledPin = 7;
bool ledState = false;          // Is the LED on or off?
bool lastButtonState = HIGH;    // Previous button reading

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);
  Serial.begin(9600);
}

void loop() {
  int currentButton = digitalRead(buttonPin);

  // Detect the moment the button goes from NOT pressed to PRESSED
  if (currentButton == LOW && lastButtonState == HIGH) {
    ledState = !ledState;  // Flip: true becomes false, false becomes true
    digitalWrite(ledPin, ledState);
    Serial.print("LED toggled: ");
    Serial.println(ledState ? "ON" : "OFF");
    delay(200);  // Simple debounce -- ignore bouncing for 200ms
  }

  lastButtonState = currentButton;
}
```

"This is a toggle! Press once, LED turns on. Press again, it turns off. The `!` symbol means NOT — it flips true to false and false to true."

**Award:** **+30 XP** **for building a toggle button!**

## Quick Quiz — Earn Bonus XP!

**Question 1:** With INPUT_PULLUP, what does the pin read when the button is NOT pressed?

- A) LOW (0V)
- B) HIGH (5V)

- C) 2.5V

**(Correct: B — +20 XP !)**

**Question 2:** What does `digitalRead(2)` return?

- A) A number from 0 to 1023
- B) Either HIGH or LOW
- C) The temperature

**(Correct: B — +20 XP !)**

**Question 3:** What does the `==` symbol mean in an if statement?

- A) "Set this value"
- B) "Is this equal to?"
- C) "Add these together"

**(Correct: B — +20 XP !)**

---

## Lesson 28 Complete!

```
==============================================

   INPUT HANDLER BADGE UNLOCKED!

   Skills unlocked:
   [check] Wire a button to Arduino
   [check] Use INPUT_PULLUP mode
   [check] Read digital inputs with digitalRead()
   [check] Write if/else decisions
   [check] Measured button voltage with the Wand
   [check] Built a toggle button

==============================================
```

**XP Breakdown:**

| Activity | XP |
|---|---|
| Hook discussion | 10 |
| Pull-up concept | 10 |
| Button circuit build | 20 |
| Button-LED code | 30 |
| Wand Check (two states) | 40 |
| Toggle challenge | 30 |
| Quiz (3 questions) | 60 |
| **TOTAL POSSIBLE** | **250** |

## Coming Up Next...

In **Lesson 29**, you move beyond just ON/OFF. You will connect a **potentiometer** and read **analog values** — numbers from 0 to 1023! The Arduino will read HOW MUCH the knob is turned, not just whether it is on or off. This is how real sensors work!

## Troubleshooting

| Problem | Fix |
|---|---|
| LED stays on all the time | Make sure the button is wired to GND on one side; check that your if statement checks for LOW |
| LED does not respond to button | Check button wiring; make sure pin 2 goes to one side and GND to the other |
| Serial Monitor shows random HIGH/LOW | Make sure you used INPUT_PULLUP, not just INPUT |
| Toggle mode misses presses | The delay(200) debounce may need adjusting; try 150 or 250 |
| Wand reads weird voltage | Make sure probes are firmly touching the pin and GND |